Neighborhood Stability in Assignments on Graphs

A report on the paper by Aziz, Lisowski, Suzuki and Vollen (arXiv:2407.05240, 2024)

Ashik Stenny (230224) Achyuth H Warrier (230052)

CS698I: Relational Structures in Games, IIT Kanpur Instructor: Sunil Simon

14 November 2026

Abstract

We study the paper *Neighborhood Stability in Assignments on Graphs* by Aziz, Lisowski, Suzuki and Vollen. Agents are placed on the vertices of a seat graph; each agent approves some other agents (binary preferences) and its utility is the number of approved neighbours. An assignment is *neighbourhood stable* if no two *adjacent* agents would both gain by swapping seats. The paper shows that such assignments need not exist (on balanced complete bipartite seat graphs), that swap dynamics can cycle even on a 4-cycle, but that a neighbourhood stable assignment always exists and can be computed in polynomial time on cycles and paths. On general seat graphs, one exists whenever the seat graph has enough leaves compared with the directed feedback vertex set number of the preference graph, and it can be computed in polynomial time when the preference graph is acyclic. We explain each result with a single running example, relate the paper to topics from the course (potential games, the finite improvement property, PLS, colouring games, hedonic games), and add our own observations: an exact characterisation of envy in the counterexample, a tighter $O(n^2)$ bound for the path algorithm, a five-agent instance showing why distance-2 stability fails on cycles, and two cases that the written proof of the main lemma for cycles does not cover, together with patches that we checked computationally.

Contents

1 Introduction and motivation **1**

2 Model **2**

2.1 Running example 3

2.2 How the notions relate 3

2.3 Why binary preferences 3

3 Neighbourhood stable assignments need not exist **3**

4 Swap dynamics can cycle **5**

5 Cycle seat graphs **6**

5.1 The swap rule 6

5.2 The case split 6

5.3 Case 1: a greedy assignment 7

5.4 Case 2: path partitions 7

5.5 Why distance-2 stability fails on cycles [ours] 9

6 Path seat graphs **10**

7 A sufficient condition for general seat graphs **10**

8 Critical remarks [ours] **11**

8.1 Two cases missing from the proof of Lemma 2 11

8.2 Theorem 4: statement versus proof 12

8.3 Running time of the path algorithm 12

8.4 Who finds the stable seating? 12

9 Connections to the course **13**

10 Open questions and conclusion **13**

A Computational verification **13**

Notation for our contributions. Statements that are ours and not in the paper are marked [**ours**]. Everything else follows the paper. Computational checks mentioned in the text are described in Appendix A.

1 Introduction and motivation

The dinner party. A host seats n guests on n fixed seats. Each guest likes some of the other guests (a yes/no opinion), and is happier the more liked guests sit next to them. If two guests sitting next to each other would *both* be happier after exchanging seats, they will swap. Can the host choose a seating in which no two neighbours want to swap?

The paper opens with an equivalent organisational story: roles are designed in advance, members do not care which project they work on but do care which colleagues they work with, and the organisation wants to avoid pairs of members who would both prefer to swap roles. The model was introduced by Bodlaender et al. (2020) as *hedonic seat arrangement* and has several recent follow-ups (Ceylan et al., Berriaud et al., Wilczynski).

Why a weaker stability notion. As in hedonic games, where agents care only about who is in their coalition, stable outcomes in seating problems frequently fail to exist; Berriaud et al. found many non-existence results even when the seat graph is a path or a cycle. In the related Schelling games, Bilò et al. showed that allowing only *local* swaps enlarges the set of equilibria, so that equilibria exist on more graph classes. The paper follows the same idea: it only forbids blocking pairs of *adjacent* agents. This is natural for seating: to agree on a swap, two guests must talk, and at a table you talk to the people next to you, who are also exactly the people affecting your utility. It is also natural when agents only know the positions of their neighbours.

The question. The paper asks: *"For which classes of seat graphs is a neighborhood stable assignment guaranteed to exist under binary preferences?"*

Summary of results.

Seat graph	Guarantee	Algorithm
$K_{t,t}$, $t \geq 3$ odd	NS may not exist (Theorem 1)	–
Cycle	NS always exists (Theorem 2)	$O(n^3)$
Path	no blocking pair within distance 2 (Theorem 3)	$O(n^2)^*$
Enough leaves ($\gamma \leq$ #leaves)	NS exists (Theorem 4)	poly. if $\mathcal{P}$ is a DAG

*The paper states $O(n^3)$; $O(n^2)$ is our analysis of the same algorithm. Here NS abbreviates *neighbourhood stable*, γ is the directed feedback vertex set (DFVS) number of the preference graph, and DAG means directed acyclic graph.

2 Model

Definition 1 (Instance)**.** A set $A = \{1, \ldots, n\}$ of agents, an undirected *seat graph* $G = (V, E)$ with $|V| = n$, and a directed *preference graph* $\mathcal{P}$ on A. We write $i \to j$ if (i, j) is an arc of $\mathcal{P}$ (agent i *approves*, or likes, agent j), and $i \nrightarrow j$ otherwise. An *assignment* is a bijection $\pi : A \to V$.

Write $N_\pi(i)$ for the agents seated next to i. The utility of agent i is the number of approved neighbours,

$$u_i(\pi) = \sum_{j \in N_\pi(i)} \mathbb{I}[i \to j].$$

Let $\pi^{i \leftrightarrow j}$ denote π with the seats of i and j exchanged. Agent i *envies* j if $u_i(\pi^{i \leftrightarrow j}) > u_i(\pi)$, and i, j form a *blocking pair* if each envies the other. An assignment is *stable* if it has no blocking pair.

Definition 2 (Neighbourhood stability)**.** An assignment π is *neighbourhood stable* (NS) if no blocking pair is assigned to adjacent vertices.

Connection to hedonic games. The paper notes that the problem is connected to hedonic games, in which agents are partitioned into coalitions according to their preferences over coalition members. **[Ours]** More precisely, utilities have the form of an additively separable hedonic game with 0/1 values, except that an agent's "coalition" is its neighbourhood in the seat graph rather than a block of a partition.

2.1 Running example

We use the paper's Example 1 (Figure 2) throughout: four agents with $a \leftrightarrow c$, $b \leftrightarrow d$, $b \to a$ and $d \to c$, seated on a path of four seats.

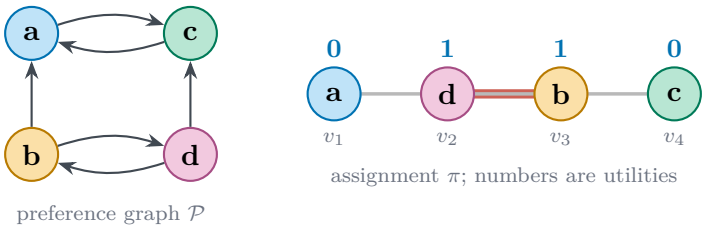


Figure 1: The running example (paper's Fig. 2). Agents b and d form an adjacent blocking pair.

Under $\pi = (a, d, b, c)$ we have $u_a = u_c = 0$ and $u_b = u_d = 1$. If b and d swap, giving (a, b, d, c), then d trades its neighbour a (not approved) for c (approved) and b trades c for a; both rise to 2. So b, d is an adjacent blocking pair and π is not NS. The assignment (a, b, d, c) is stable, hence also NS.

2.2 How the notions relate

Stable $\Rightarrow$ no blocking pair within distance 2 (the notion studied by Berriaud et al.) $\Rightarrow$ NS. Deciding whether a stable assignment exists is computationally hard; the paper derives this from the exchange-stable roommates problem, which is NP-complete.

Example 1 (NS is genuinely weaker; paper's Example 2, from Massand and Simon, IJCAI 2019)**.** Let the seat graph be two disjoint triangles and the preference graph a directed 6-cycle $a \to b \to c \to d \to e \to f \to a$. No assignment is stable: in every assignment there are two agents at different triangles who both have utility 0, and they block. Yet every assignment is NS, because adjacent agents share a triangle, and swapping two agents of a triangle leaves every neighbourhood unchanged. (We confirmed both facts over all 720 assignments.)

2.3 Why binary preferences

Richer preferences fail almost immediately. With three non-negative values, there are four agents with no NS assignment on a cycle (the paper adapts the proof of Theorem 8 of Berriaud et al.). With values ± 1, the paper notes that "it is easy to construct a counterexample of neighborhood stability on a path of length three" by letting the friendship graph be a cycle. **[Ours]** One such instance: three agents on a path of three seats, each valuing its successor on a directed 3-cycle at $+1$ and the other agent at -1; we checked that none of the six seatings is NS. Two non-negative values reduce to binary preferences without loss of generality.

3 Neighbourhood stable assignments need not exist

Theorem 1. *For every $n = 2t$ with $t \geq 3$ odd, there is an instance with n agents and no NS assignment.*

Construction. The seat graph is $K_{t,t}$ with sides L and R. Agents form a top row $s_{11}, \ldots, s_{1t}$ and a bottom row $s_{21}, \ldots, s_{2t}$; indices are taken modulo t. Each agent approves exactly two agents: its *twin* ($s_{1j} \leftrightarrow s_{2j}$) and its *row successor* ($s_{ij} \to s_{i,j+1}$). So $\mathcal{P}$ consists of t directed 2-cycles and two directed t-cycles (Figure 2).

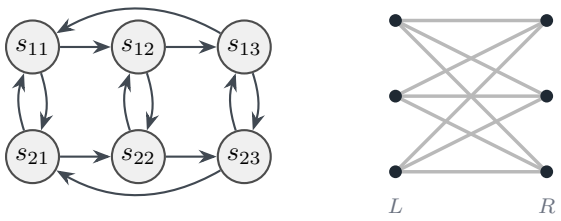


Figure 2: Theorem 1 for $t = 3$ (paper's Fig. 1): preference graph (left) and seat graph $K_{3,3}$ (right).

Key simplification. In $K_{t,t}$ the neighbours of an agent are exactly the agents on the other side, so an assignment is determined (for utilities) by the split into sides L and R. Each agent approves exactly two agents, so its utility is the number of its two favourites on the other side. If i swaps with j on the other side, i's new neighbours are its former side-mates together with j. The paper uses two sufficient conditions for i to envy an agent j on the other side:

(C1) both of i's favourites are on i's own side ($u_i = 0$); then i envies *every* agent on the other side ($0 \to 2$);

(C2) exactly one favourite is on i's own side and $i \to j$; then i envies j ($1 \to 2$).

Lemma 1 (ours). *In this instance, i envies j if and only if (C1) or (C2) holds.*

Proof. If i, j are on the same side, swapping changes nothing for i. Otherwise let k be the number of i's favourites on its own side. Then $u_i(\pi) = 2 - k$ and $u_i(\pi^{i \leftrightarrow j}) = k + \mathbb{I}[i \to j]$. For $k = 2$ this is $2 > 0$ (C1); for $k = 1$ it is $1 + \mathbb{I}[i \to j] > 1$ iff $i \to j$ (C2); for $k = 0$ the agent already has the maximum utility 2. □

A blocking pair needs envy in both directions, and each direction may use either condition. Since agents on opposite sides are adjacent, any blocking pair across the sides violates NS. The proof considers two cases.

Case 1: some twin pair sits on one side.

1. A split twin pair contributes one agent to each side. Since both sides have t agents, the together pairs must be shared equally: *L and R contain the same number of together pairs.* Hence the number of agents in together pairs is even, and as t is odd, at least one twin pair is split.
2. Going around the indices, some together pair at index j is followed by a split pair at $j+1$. Relabel so that $s_{1j}, s_{2j}, s_{1,j+1} \in L$ and $s_{2,j+1} \in R$.
3. Both favourites of s_{1j} are in L, so by (C1) it envies everyone in R.
4. If some agent of R had utility 0, it would envy s_{1j} by (C1) and they would block. So every agent of R approves someone in L. For a together pair in R at index l, the agent s_{1l} approves $s_{2l} \in R$, so $s_{1,l+1}$ must be in L; likewise $s_{2,l+1} \in L$. Thus every together pair in R at l is followed by a together pair in L at $l+1$.
5. This map from together pairs in R to together pairs in L is injective, and both sets have the same size, so it is a bijection. In particular the pair at $j-1$ is together in R.
6. Then $s_{1,j-1} \in R$ approves $s_{2,j-1}$ (own side) and s_{1j} (other side), so it envies s_{1j} by (C2). Together with step 3, $s_{1,j-1}$ and s_{1j} block.

Case 2: every twin pair is split. Side L contains one agent of every twin pair, so at least $\lceil t/2 \rceil$ of its agents come from the same row, say the top row. These agents lie on the directed t-cycle $s_{11} \to \cdots \to s_{1t} \to s_{11}$, whose largest independent set has size $\lfloor t/2 \rfloor < \lceil t/2 \rceil$ because t is odd. Hence two consecutive top agents, say s_{11}, s_{12}, are in L, and their twins s_{21}, s_{22} are in R. Now s_{11} approves s_{12} (own side) and s_{21} (other side), and s_{21} approves s_{22} (own side) and s_{11} (other side). By (C2) they envy each other.

The case $t = 3$ in detail [ours]. Up to exchanging the sides there are 10 splits. If some twins are together, each side has at most one together pair (two would need four agents), so Case 1 means *one together pair on each side and one split pair*; there are 6 such splits. For example,

take $L = \{s_{11}, s_{21}, s_{12}\}$ and $R = \{s_{22}, s_{13}, s_{23}\}$. Twins 1 are together and twins 2 are split, so $j = 1$; s_{11} has utility 0. The only remaining index is $3 = j - 1$, whose twins are together in R, and s_{13} has utility 1. Swapping s_{11} and s_{13} raises their utilities from 0 to 2 and from 1 to 2. The remaining 4 splits are Case 2; for instance $L = \{s_{11}, s_{12}, s_{23}\}$, $R = \{s_{21}, s_{22}, s_{13}\}$, where s_{11} and s_{21} both go from 1 to 2 by swapping. In a triangle every two vertices are consecutive, which is why Case 2 is immediate for $t = 3$. We confirmed by brute force that none of the 720 assignments is NS.

Where oddness is used. Case 1 needs a split twin pair, and Case 2 needs an odd cycle. **[Ours]** For even t the construction does not work: for $t = 4$, putting $s_{11}, s_{13}, s_{22}, s_{24}$ on L places every agent's twin and successor on the other side, so every agent has the maximum utility 2 and the assignment is stable.

4 Swap dynamics can cycle

Swap dynamics starts from an arbitrary assignment and lets adjacent blocking pairs swap until none remains.

Proposition 1. *There is an instance and an assignment from which swap dynamics never converge, even when the seat graph is a cycle of length* 4.

The instance (paper's Fig. 6) has $c \to a$, $a \to b$, $a \to d$, $b \to d$, $d \to c$. Figure 3 shows the four swaps. In each assignment the highlighted pair is the *unique* adjacent blocking pair, so there is no choice of swap at any step.

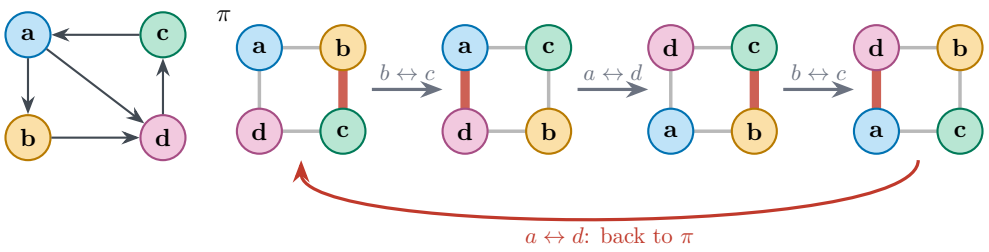


Figure 3: Proposition 1. Left: preferences. Right: the cycle of swaps on a 4-cycle seat graph; red marks the unique blocking pair. Gains: b, c go $0 \to 1$; a goes $1 \to 2$ and d goes $0 \to 1$.

Consequences and course connections.

- The four swaps form an *improvement cycle*. In a finite game, the finite improvement property (FIP) is equivalent to having a generalised ordinal potential (Monderer and Shapley). By analogy, treating each blocking swap as a move, no function can strictly increase with every blocking swap on cycles.
- Because the blocking pair is unique at every step, *every* improvement path from π cycles: the dynamics is not even weakly acyclic **[ours]**.
- Nevertheless the instance has NS assignments (8 of the 24 seatings, by our count). Existence on cycles therefore needs a constructive argument, which is what Theorem 2 provides.

When dynamics do converge. The paper remarks that symmetric preferences ($i \to j$ iff $j \to i$) guarantee NS, analogously to Bullinger and Suksompong. **[Ours]** A short potential

argument: let $F(\pi)$ be the number of seat edges whose two endpoints approve each other. With symmetric preferences, $u_i(\pi)$ equals the number of such edges at i's seat. When i and j swap, only edges at their two seats change, and the edge between them (if any) keeps the same two endpoints. Hence $F(\pi^{i\leftrightarrow j}) - F(\pi) = \Delta u_i + \Delta u_j > 0$ for every blocking swap, adjacent or not. Since $F \le |E|$, swap dynamics terminates in an NS assignment; if non-adjacent blocking pairs are also allowed to swap, it terminates in a *stable* assignment.

5 Cycle seat graphs

Theorem 2 (central result)**.** *If the seat graph is a cycle, an NS assignment always exists and can be computed in $O(n^3)$ time.*

The paper calls this surprising: on a cycle, forbidding blocking pairs within distance 2 can fail with five agents (Berriaud et al.), three-valued preferences fail with four agents, and swap dynamics may cycle (Proposition 1).

5.1 The swap rule

On a cycle C_n with $n \ge 4$, let x sit at v_i and y at v_{i+1}, and let p and z be the agents at v_{i-1} and v_{i+2}. After swapping, x and y are still adjacent; x loses p and gains z, and y loses z and gains p. Therefore

$$x \text{ envies } y \iff x \to z \text{ and } x \not\to p, \qquad y \text{ envies } x \iff y \to p \text{ and } y \not\to z.$$

The paper's Observation 2 is the statement that if $x \to p$ then x does not envy y; Observation 1 states that if $y \not\to p$ or $x \not\to z$, then x, y cannot block. In the running example, $(x, y, p, z) = (d, b, a, c)$ satisfies all four conditions, which is why b, d block.

5.2 The case split

Case 1 (a "V"). There are s, t, w with $s \to w \leftarrow t$ and no arc between s and t in either direction.

Case 2 (no "V"). Whenever two agents approve the same agent, one of them approves the other.

The running example is in Case 1 ($b \to a \leftarrow c$ and b, c unrelated). The instance of Proposition 1 is in Case 2: only d has two admirers, a and b, and $a \to b$. Checking all triples decides the case in $O(n^3)$ time.

5.3 Case 1: a greedy assignment

Seat s, w, t on v_1, v_2, v_3. For $i = 4, \dots, n$, seat at v_i an unseated agent who approves the agent at v_{i-1} if one exists, and otherwise any unseated agent.

Correctness. By the swap rule, an adjacent pair x, y (with p left of x) can block only if $y \to p$ and $x \not\to p$.

- Around the V: $s \not\to t$, so s does not envy w; $t \not\to s$, so t does not envy w. Since $s \to w$ and $t \to w$, neither s nor t envies its other neighbour (Observation 2).

- Elsewhere, take x at v_i and y at v_{i+1} with $i \geq 4$. If $x \to p$, then x does not envy y. If $x \not\to p$, then when x was chosen no unseated agent approved p; y was unseated then, so $y \not\to p$ and y does not envy x.

So the greedy satisfies the invariant "for every adjacent pair, either x approves the agent on its left or y does not", which is all NS needs. The construction takes $O(n^2)$ time.

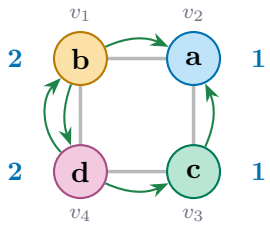


Figure 4: Case 1 on the running example with $s = b$, $w = a$, $t = c$; only d approves c, so d takes v_4. Green arrows are approvals between neighbours. Every agent attains its maximum possible utility.

5.4 Case 2: path partitions

Definition 3. A *path partition* of $\mathcal{P}$ is a collection of directed paths $P_1, \ldots, P_k$ covering every agent exactly once. It is *minimal* if $\text{tail}(P_i) \not\to \text{head}(P_j)$ for all $i \neq j$. The assignment $\Phi(P_1, \ldots, P_k)$ seats the paths consecutively around the cycle (Figure 5).

Inside a path every agent approves its right neighbour, so only path boundaries can contain agents that do not. Let $R(\pi)$ be the number of agents that approve their right neighbour; for a minimal partition with $k \geq 2$ paths, $R = n - k$. The initial minimal partition is obtained by repeatedly removing a maximal path: if a tail approved another path's head, one of the two paths could have been extended when it was built.

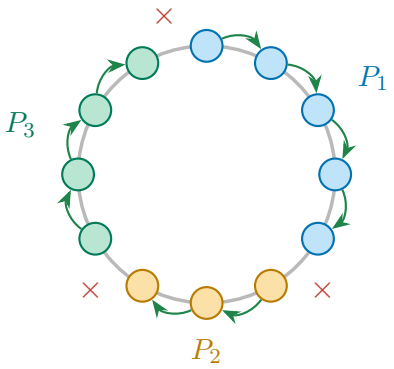


Figure 5: $\Phi(P_1, P_2, P_3)$: paths laid around the cycle (cf. paper's Fig. 4). Green: approval of the right neighbour; red crosses: path boundaries.

Claim 1 (insertion)**.** *In Case 2, if $s \notin P = s_1 \to \cdots \to s_r$ and $s \to s_r$, then $s \to s_1$ or $s_j \to s \to s_{j+1}$ for some j.*

Idea. s and s_{r-1} both approve s_r, so (no V) one approves the other. If $s_{r-1} \to s$, insert s between s_{r-1} and s_r; otherwise $s \to s_{r-1}$ and we repeat one step back. Formally this is an induction on r. This is the only place where the Case 2 assumption is used.

Lemma 2. *Let $\pi = \Phi(P_1, \ldots, P_k)$ for a minimal path partition. If π has an adjacent blocking pair, there is a minimal path partition whose assignment ρ satisfies $R(\rho) \geq R(\pi) + 1$.*

Let the blocking pair be x, y at v_i, v_{i+1}, with p at v_{i-1} and z at v_{i+2}. There are two types.

Type I: x, y consecutive in the same path P_r. The swap rule requires $y \nrightarrow z$; inside a path y approves its successor, so $y = \text{tail}(P_r)$ and $z = \text{head}(P_{r+1})$. The rule also gives $y \to p$ and $x \to z$. If $|P_r| \geq 3$: remove x, y from P_r; the rest ends at p, and since $y \to p$, Claim 1 re-inserts y without changing the tail p. Then $p \to x \to z$ glues the result onto P_{r+1} (Figure 6). If $|P_r| = 2$: insert y into P_{r-1} (as $y \to p = \text{tail}(P_{r-1})$) and prepend x to P_{r+1}. Either way the number of paths drops; gluing further where possible restores minimality, and R increases.

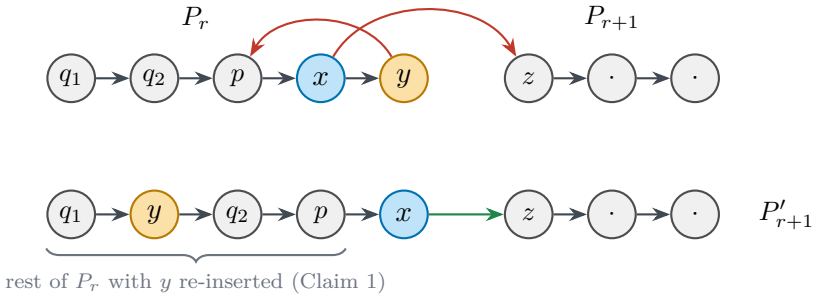


Figure 6: Type I repair with $|P_r| \geq 3$ (cf. paper's Fig. 5). Red arcs are forced by the blocking pair.

Type II: $x = \text{tail}(P_r)$, $y = \text{head}(P_{r+1})$. The swap rule requires $y \nrightarrow z$. If $|P_{r+1}| \geq 2$, then y approves its successor z; so $P_{r+1} = \{y\}$ and $z = \text{head}(P_{r+2})$. The rule also requires $x \to z$, i.e. $\text{tail}(P_r) \to \text{head}(P_{r+2})$, which contradicts minimality. So Type II cannot occur. (See Section 8 for two boundary cases.)

Termination and running time. $R \leq n$ and each application of Lemma 2 increases R, so at most n repairs occur, and the final assignment has no adjacent blocking pair. The paper states that each repair takes $O(n^2)$ time. Our reconstruction: finding an adjacent blocking pair is $O(n)$ (four look-ups per pair by the swap rule); the insertion of Claim 1 walks back along one path in $O(n)$; gluing lists is $O(n)$; restoring minimality checks tail–head pairs, at most $O(k^2) = O(n^2)$, and each glue only creates one new tail and head to recheck. Overall:

Step	Cost
Decide the case (all triples)	$O(n^3)$
Case 1 greedy	$O(n^2)$
Case 2: initial minimal partition	$O(n^2)$
Case 2: at most n repairs of $O(n^2)$ each	$O(n^3)$

[Ours] The case check as described (examining triples) needs $\Theta(n^3)$ time in the worst case: when no V exists it must examine every pair of in-neighbours of every agent, and for a transitive tournament this is about $n^3/6$ pairs. Whether a faster algorithm exists is not addressed in the paper.

Back to Proposition 1. The looping instance is in Case 2. If the algorithm's first maximal path is $c \to a \to b \to d$ (which covers every agent, and $d \to c$), then wrapped around the cycle

every agent approves its right neighbour ($R = 4$) and the assignment is NS at once, although swap dynamics from π never reaches an NS assignment. A different choice of maximal paths can start from the partition $\{a \to d \to c,\ b\}$ instead; this case is discussed in Section 8.

Course connection. R is a progress measure in the spirit of the energy of Hopfield networks. Unlike the potentials behind PLS (Polynomial Local Search) problems, it takes at most $n + 1$ values, so the number of improving steps is polynomial. Note that the steps are re-arrangements made by the algorithm, not swaps made by agents.

5.5 Why distance-2 stability fails on cycles [ours]

On paths the paper obtains the stronger guarantee of no blocking pair within distance 2 (Theorem 3); on cycles this is impossible by the five-agent instance of Berriaud et al. The paper does not reproduce their instance; we found the following one by computer search. On C_5 every two seats are at distance 1 or 2, so distance-2 stability coincides with stability. Let

$$a \to c, e; \quad b \to a, e; \quad c \to a, d; \quad d \to b, c; \quad e \to b, d,$$

i.e. mutual pairs $a \leftrightarrow c$, $c \leftrightarrow d$, $b \leftrightarrow e$ and the directed 4-cycle $a \to e \to d \to b \to a$. Up to rotation and reflection there are 12 seatings, and each has a blocking pair:

Seating	Pair	Distance	Gains	NS?
$a\,b\,c\,d\,e$	c, e	2	$1 \to 2$, $1 \to 2$	yes
$a\,b\,c\,e\,d$	a, c	2	$0 \to 1$, $0 \to 1$	no
$a\,b\,d\,c\,e$	a, e	1	$1 \to 2$, $0 \to 1$	no
$a\,b\,d\,e\,c$	b, c	2	$1 \to 2$, $1 \to 2$	yes
$a\,b\,e\,c\,d$	a, e	2	$0 \to 1$, $1 \to 2$	yes
$a\,b\,e\,d\,c$	a, d	2	$1 \to 2$, $1 \to 2$	yes
$a\,c\,b\,d\,e$	b, d	1	$0 \to 1$, $1 \to 2$	no
$a\,c\,b\,e\,d$	b, d	2	$1 \to 2$, $0 \to 1$	yes
$a\,c\,d\,b\,e$	b, e	1	$1 \to 2$, $1 \to 2$	no
$a\,c\,e\,b\,d$	a, c	1	$1 \to 2$, $1 \to 2$	no
$a\,d\,b\,c\,e$	a, e	1	$1 \to 2$, $0 \to 1$	no
$a\,d\,c\,b\,e$	c, d	1	$1 \to 2$, $1 \to 2$	no

Five of the seatings are NS, as Theorem 2 requires; their only blocking pairs are at distance 2.

The path proof does not transfer because it uses two features a cycle lacks: the first seat is an endpoint (anyone swapping into it loses a neighbour), and the greedy guarantee runs in one direction only, so on a cycle the pairs around the point where the last seat meets the first are unconstrained. Case 1 above seats a V first precisely to make that junction safe for adjacent swaps; distance 2 would require control two seats away on both sides.

6 Path seat graphs

Theorem 3. *If the seat graph is a path, an assignment with no blocking pair within distance 2 exists and can be computed in $O(n^2)$ time. (The paper states $O(n^3)$; the bound $O(n^2)$ is our analysis of the same algorithm.)*

Algorithm. Seat any agent on v_1. For $k = 2, \ldots, n$, seat on v_k an unseated agent who approves the agent on v_{k-1} if one exists, and otherwise any unseated agent. Each of the n steps scans at most n unseated agents with one look-up each, giving $O(n^2)$.

Correctness. The agents on v_2 and v_3 never envy the agent on v_1: after such a swap their neighbourhood is a subset of their current one. For $i \geq 2$, let x be the agent on v_i.

- If x approves its left neighbour, then after a swap with v_{i+1} or v_{i+2} it can gain only by reaching utility 2, which requires approving the agent on v_{i+1} (a neighbour in both swapped positions); but then x already has utility 2.
- If x does not approve its left neighbour, then by construction neither the agent on v_{i+1} nor the one on v_{i+2} approves it (otherwise it would have been chosen instead of x). After swapping with x, the only new neighbour of either agent is that left neighbour, so neither gains.

Running example. Starting from a: $c \to a$, then $d \to c$, then $b \to d$, giving (a, c, d, b) (Figure 7), which is in fact stable.

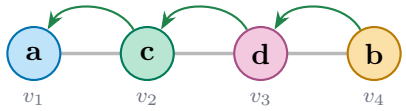


Figure 7: Theorem 3 on the running example: each new agent approves the previous one.

Context and course connection. Berriaud et al. showed by a potential argument that swap dynamics on paths converge to an assignment with no blocking pair within distance 2, gave evidence that convergence may take exponentially many steps, and left open whether such an assignment can be computed in polynomial time. This is the PLS phenomenon from the course: improvement must stop, but possibly only after exponentially many steps. The paper settles the question by a direct construction that does not simulate the dynamics.

7 A sufficient condition for general seat graphs

Definition 4. A set $X \subseteq A$ is a *directed feedback vertex set* (DFVS) if $\mathcal{P}[A \setminus X]$ is acyclic; the DFVS number γ is the minimum size of such a set. A *leaf* of G is a vertex of degree one.

Theorem 4. *If the number of leaves of G exceeds the DFVS number γ of $\mathcal{P}$, there exists an NS assignment on G. When $\gamma = 0$, i.e. $\mathcal{P}$ is a DAG, a stable assignment can be computed in polynomial time.*

The proof only uses that a DFVS fits on the leaves, i.e. $\gamma \leq$ #leaves (Section 8).

Sinks. A *sink* of a directed graph is a vertex with no outgoing arcs. Every DAG has one: following arcs from any vertex can never revisit a vertex, so the walk ends at a vertex with no outgoing arc. Removing a sink leaves a DAG, so the agents of $A \setminus X$ can be ordered $s_1, s_2, \ldots$ with s_ℓ a sink of $\mathcal{P}[A \setminus (X \cup \{s_1, \ldots, s_{\ell-1}\})]$. Consequently everyone approved by s_i lies in $X \cup \{s_1, \ldots, s_{i-1}\}$.

Algorithm.

1. Seat the agents of a smallest DFVS X on leaves.
2. In the order $s_1, s_2, \ldots$, let each agent take the free seat with the largest number of already-seated approved neighbours (ties broken arbitrarily).

Correctness. Take $i < j$. When s_i chose, all agents it approves were seated and never move afterwards, and the seat of s_j was still free. So s_i judged that seat at most as good, and after a swap its approved neighbours there are the same (it does not approve s_j). Hence s_i never envies a later agent, and no blocking pair (at any distance) consists of two agents outside X. Now let $x \in X$ sit on a leaf with unique neighbour y. If y swaps onto the leaf, its only neighbour becomes x, whom it already had, so y does not envy x. Hence there is no adjacent blocking pair. If $X = \emptyset$ there is no blocking pair at all.

The seat choice matters [ours]. On the running example with seats $v_1v_2v_3v_4$ (a path), the two 2-cycles $a \leftrightarrow c$, $b \leftrightarrow d$ are disjoint, so $\gamma = 2$; take $X = \{a, b\}$ on the leaves v_1, v_4. Among c, d we have $d \to c$, so c is the sink and chooses first; it approves a, so it takes v_2, and d takes v_3. The result (a, c, d, b) is NS (indeed stable). Had c taken v_3 instead, the result (a, d, c, b) gives $u_c = 0$ and $u_d = 1$, and swapping c, d raises them to 1 and 2: an adjacent blocking pair. Note also that this example has $\gamma = 2 = \#\text{leaves}$, which the proof covers but the literal statement ("exceeds") does not.

Consequence. Planar directed graphs have DFVS number at most $\frac{3}{5}n$ (via Borodin's acyclic colouring theorem), so for planar preference graphs an NS assignment exists on every seat graph with at least 60% leaves, for example full m-ary trees with $m \geq 3$, which have at least $\frac{m-1}{m}n$ leaves.

Course connection [ours]. The argument mirrors the existence argument for colouring games on DAGs: agents choose in reverse topological order, sinks first, so no later choice can undermine an earlier one. Here the DAG is the *preference* graph; in Theorem 2 the cycle is the *seat* graph.

8 Critical remarks [ours]

8.1 Two cases missing from the proof of Lemma 2

The Type II argument derives $\text{tail}(P_r) \to \text{head}(P_{r+2})$ and appeals to minimality, which only forbids $\text{tail}(P_i) \to \text{head}(P_j)$ for $i \neq j$. The Type I argument glues the repaired path onto P_{r+1} and counts on the number of paths dropping. Both steps break when there are very few paths (indices are taken modulo k).

Gap 1: Type II with $k = 2$. Then $P_{r+2} = P_r$, and $x \to z$ says that P_r's tail approves its own head, which minimality allows. *Example:* in the instance of Proposition 1, the partition $\{a \to d \to c,\ b\}$ is minimal ($c \nrightarrow b$ and $b \nrightarrow a$), and Φ seats a, d, c, b around the cycle. With $(x, y, p, z) = (c, b, d, a)$ we have $c \to a$, $c \nrightarrow d$, $b \to d$, $b \nrightarrow a$: a Type II blocking pair.

Patch. In this case P_r closes into a directed cycle (its tail approves its head) and $y \to p$ with $p \in P_r$ the predecessor of x. Cut the cycle just before p and prepend y: $y \to p \to x \to z \to \cdots$ is a single path covering all agents, so k drops from 2 to 1 and R increases. In the example this gives $b \to d \to c \to a$.

Gap 2: Type I with $k = 1$. Then $P_{r+1} = P_r$, so "glue onto P_{r+1}" is not available and the number of paths cannot drop below one. *Example:* $a \to b$, $a \to c$, $b \to c$, $b \to d$, $c \to d$, $d \to a$ (Case 2). The maximal path $b \to d \to a \to c$ seated around the cycle has the Type I blocking pair $x = a$, $y = c$ (with $p = d$, $z = b$).

Patch. Remove x, y; re-insert y by Claim 1. Since $y \nrightarrow z$, y is not prepended, so the path still starts at z and ends at p. Appending x gives a path ending in x with $x \to z$, a Hamiltonian cycle of $\mathcal{P}$: R rises from $n-1$ to n, and an assignment in which every agent approves its right neighbour is NS: in any adjacent pair, the right agent approves its own right neighbour, so by the swap rule it does not envy the left agent. In the example the result is $b \to c \to d \to a \to b$.

Evidence. We implemented Algorithm 1 as written in the paper together with both patches and ran it on 60,000 random instances with 4 to 9 agents (17,592 of them in Case 2). Gap 1 occurred 106 times and Gap 2 644 times; the paper's contradiction for Type II with three or more paths was never violated; R increased at every repair; and every output was NS. Independently, every one of the 4096 preference graphs on four agents admits an NS assignment on C_4. We therefore believe Theorem 2 is correct and the gaps are in the write-up only.

8.2 Theorem 4: statement versus proof

- The statement requires the number of leaves to *exceed* γ, but the proof uses only $|X| \leq |L|$, and the introduction says "upper bounded by". The running example ($\gamma = 2 = \#\text{leaves}$) is covered by the proof but not by the statement.
- The abstract and introduction speak of a polynomial-time algorithm whenever the DFVS is at most the number of leaves, while the theorem claims polynomial time only for $\gamma = 0$. For $\gamma > 0$ the algorithm needs a small enough DFVS, and finding a minimum DFVS is NP-hard in general (Karp, 1972); the paper does not explain how to find X efficiently.

8.3 Running time of the path algorithm

As shown in Theorem 3, the greedy runs in $O(n^2)$, which is tighter than the stated $O(n^3)$.

8.4 Who finds the stable seating?

The motivation is agents arranging swaps with their neighbours, but on cycles such swaps may never reach an NS assignment (Proposition 1). The positive results therefore guarantee what a central planner can compute, not where decentralised behaviour leads. On paths the dynamics do converge (Berriaud et al.), but possibly slowly.

9 Connections to the course

Course topic	Where it appears
Hedonic games	Utility is additively separable with 0/1 values, over the seat neighbourhood rather than a coalition.
Potential games, FIP, generalised ordinal potentials	Proposition 1 is an improvement cycle, so no potential exists for swap dynamics on cycles; with symmetric preferences the number of mutually approving adjacent pairs is a potential.
Weakly acyclic games	From the assignment of Proposition 1, every improvement path cycles.
Hopfield networks, progress measures	The count R in Lemma 2 rises with each repair and takes at most $n + 1$ values.
PLS	On paths, swap dynamics converge but possibly in exponential time; Theorem 3 constructs the outcome directly.
Colouring games on DAGs	Theorem 4 lets sinks choose first, as in the existence proof for DAGs.

10 Open questions and conclusion

The paper lists the following directions: other seat graphs such as grids and trees, and more conditions in the style of Theorem 4; efficiency, i.e. bounding the price of neighbourhood stability or maximising welfare subject to NS; and preferences beyond binary, where cycles already fail with cardinal values and paths remain open.

To conclude: restricting swaps to neighbours turns a notion that frequently fails into one that is guaranteed on cycles and paths, although it can still fail on dense seat graphs such as $K_{t,t}$. The main technical idea, laying out path partitions of the preference graph around the cycle and repairing blocking pairs by merging paths, gives a polynomial algorithm in a setting where the natural dynamics can cycle forever. Our checks support all the paper's results, and identify two small cases in the main lemma's proof that need the patches above.

A Computational verification

To guard against errors in our own reading, we checked the small examples and our additional claims with short Python programs written for this report. Agents, seat graphs and approvals are stored explicitly; utilities and blocking pairs are computed directly from the definitions in Section 2.

Exhaustive checks.

- Running example: b, d is the only adjacent blocking pair of (a, d, b, c); (a, b, d, c) is stable; the Case 1 output (b, a, c, d) on C_4 is NS; (a, c, d, b) on the path is stable; (a, d, c, b) has the adjacent blocking pair c, d.
- Proposition 1: the four swaps return to π, the blocking pair is unique at every step, and 8 of the 24 seatings are NS.
- Theorem 1 for $t = 3$: none of the 720 assignments on $K_{3,3}$ is NS. For $t = 4$ the assignment given in the text gives every agent utility 2.

- Example 2: all 720 assignments are NS and none is stable.
- The ± 1 example on a path of three seats: no seating is NS.
- Theorem 2 for $n = 4$: each of the 4096 preference graphs on four agents has an NS assignment on C_4.
- The five-agent instance of Section 5.5: no stable seating on C_5; 10 of the 24 seatings with a fixed on one seat are NS (5 up to reflection).
- The two examples of Section 8: each has the stated blocking pair, and the patched assignment is NS.

Randomised checks.

- Theorem 3: 20,000 random instances with 3 to 8 agents; the greedy output never had a blocking pair within distance 2.
- Theorem 2: Algorithm 1 implemented as in the paper, with the two patches of Section 8, on 60,000 random instances with 4 to 9 agents. At every repair the program asserted the properties used in the proof (the type of the blocking pair, the arcs forced by the swap rule, the success of Claim 1, and the increase of R); all outputs were NS.

These checks support the statements but are not proofs; for larger instances we rely on the arguments given in the text.

References

[1] H. Aziz, G. Lisowski, M. Suzuki and J. Vollen. Neighborhood stability in assignments on graphs. arXiv:2407.05240, 2024.

[2] D. Berriaud, A. Constantinescu and R. Wattenhofer. Stable dinner party seating arrangements. WINE 2023; arXiv:2305.09549.

[3] H. L. Bodlaender, T. Hanaka, L. Jaffke, H. Ono, Y. Otachi and T. C. van der Zanden. Hedonic seat arrangement problems. AAMAS 2020.

[4] D. Bilò, V. Bilò, P. Lenzner and L. Molitor. Topological influence and locality in swap Schelling games. Autonomous Agents and Multi-Agent Systems 36(2), 2022.

[5] M. Bullinger and W. Suksompong. Topological distance games. Theoretical Computer Science 981, 2024.

[6] S. Massand and S. Simon. Graphical one-sided markets. IJCAI 2019.

[7] O. V. Borodin. On acyclic colorings of planar graphs. Discrete Mathematics 25(3), 1979.

[8] D. Monderer and L. S. Shapley. Potential games. Games and Economic Behavior 14(1), 1996.

[9] R. M. Karp. Reducibility among combinatorial problems. In Complexity of Computer Computations, 1972.